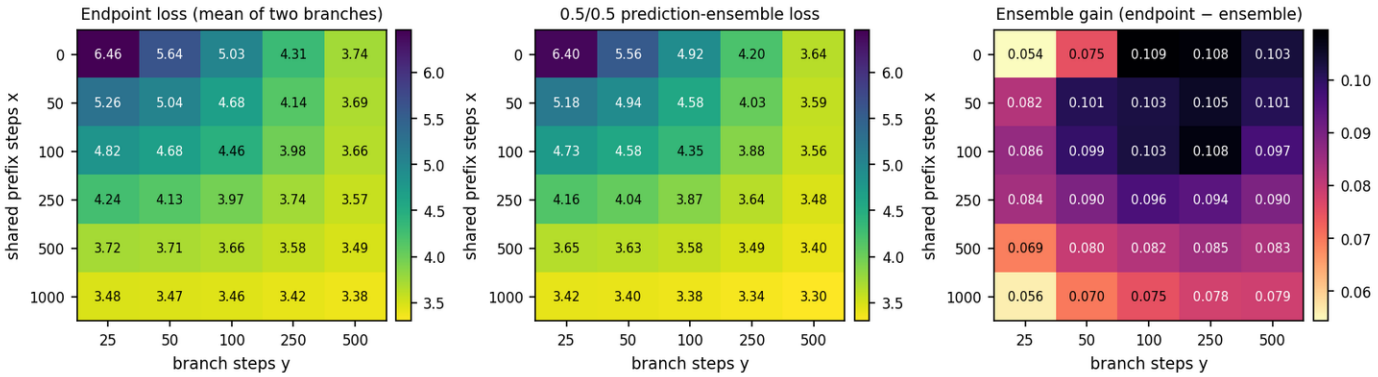
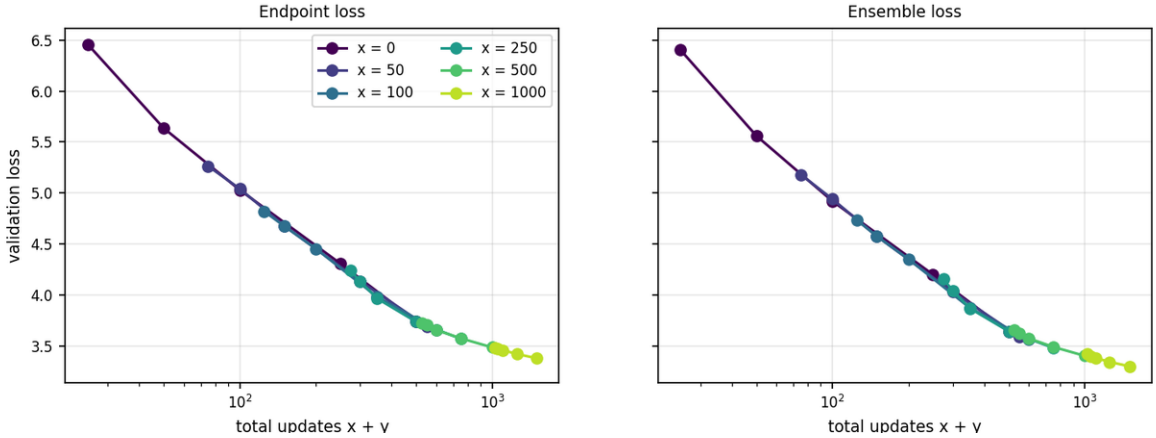
Assignment 3: Optimization 2 (Kesavan Ramakrishnan)

Problem 1: Mode connectivity

Midpoint barrier $B(\theta 1, \theta 2) = L((\theta 1+\theta 2)/2) - (L(\theta 1)+L(\theta 2))/2$; weight averaging works when $B \leq 0.01$. Setup: d8, 614M-token schedule, constant LR after warmup, AdamW. Prefix of x steps, two branches (batch-order seeds 1, 2) of 500 more steps; $x \in \{0, 50, 100, 250, 500, 1000\}$, $y \in \{25, 50, 100, 250, 500\}$.

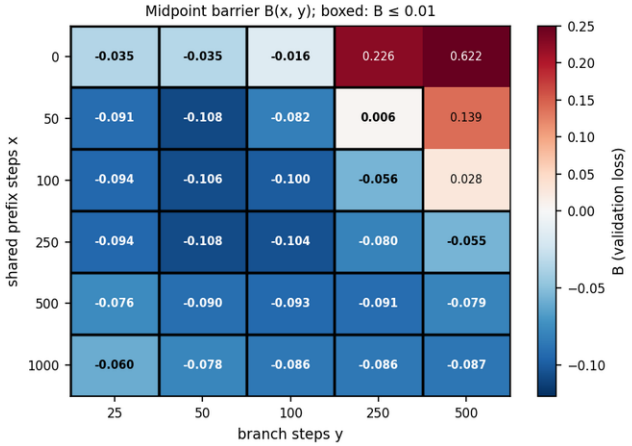
(a) Endpoint and ensemble losses





- **Endpoints depend on total updates only.** The curves for every x fall on one line in x + y, so forking at x and training y further steps costs nothing relative to a single run of x + y steps. The two branches of each cell end within 0.03 of each other.
- **The ensemble always beats the endpoints**, by 0.054 to 0.109 in loss (mean of the two endpoints minus the ensemble). The gain is largest at moderate y (50–250) for $x \leq 250$ (about 0.10) and smallest at the grid's edges: the very first steps ($x = 0$, $y = 25$: 0.054) and the late, long-prefix cells ($x = 1000$: 0.056–0.079).

(b) Weight averaging and the barrier

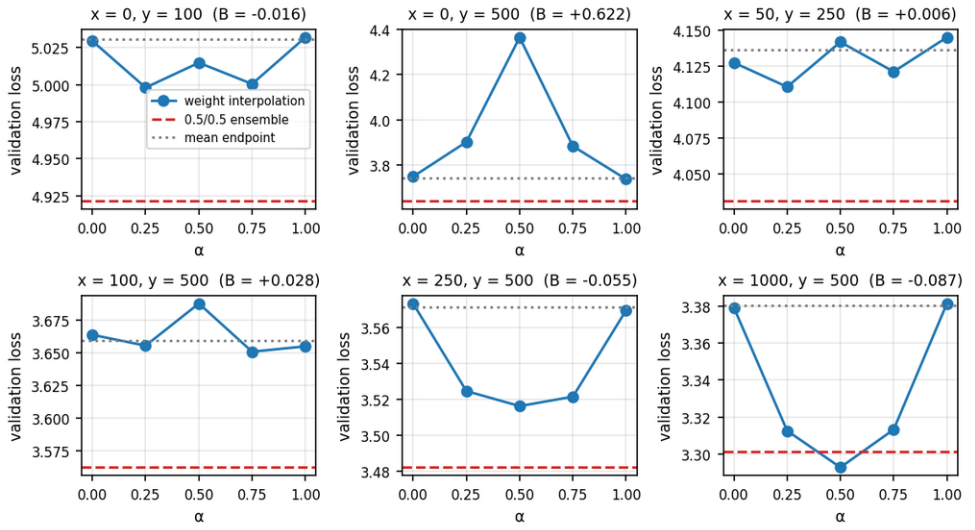


x	y	endpoint 1	endpoint 2	average	ensemble	B(x, y)	$B \leq 0.01$
0	25	6.4534	6.4633	6.4232	6.4040	-0.0352	yes
0	50	5.6239	5.6485	5.6017	5.5614	-0.0345	yes

x	y	endpoint 1	endpoint 2	average	ensemble	B(x, y)	B ≤ 0.01
0	100	5.0295	5.0319	5.0148	4.9213	-0.0159	yes
0	250	4.3130	4.3070	4.5359	4.2015	+0.2259	no
0	500	3.7489	3.7371	4.3647	3.6398	+0.6217	no
50	25	5.2575	5.2702	5.1728	5.1816	-0.0911	yes
50	50	5.0487	5.0361	4.9343	4.9412	-0.1081	yes
50	100	4.6793	4.6801	4.5977	4.5771	-0.0820	yes
50	250	4.1274	4.1450	4.1418	4.0308	+0.0056	yes
50	500	3.6995	3.6887	3.8335	3.5929	+0.1394	no
100	25	4.8191	4.8221	4.7266	4.7343	-0.0940	yes
100	50	4.6714	4.6807	4.5698	4.5769	-0.1063	yes
100	100	4.4649	4.4463	4.3554	4.3527	-0.1002	yes
100	250	3.9952	3.9705	3.9267	3.8751	-0.0562	yes
100	500	3.6636	3.6548	3.6875	3.5624	+0.0283	no
250	25	4.2365	4.2430	4.1457	4.1554	-0.0940	yes
250	50	4.1315	4.1352	4.0257	4.0438	-0.1077	yes
250	100	3.9699	3.9635	3.8622	3.8705	-0.1045	yes
250	250	3.7405	3.7347	3.6575	3.6433	-0.0801	yes
250	500	3.5736	3.5698	3.5163	3.4820	-0.0554	yes
500	25	3.7257	3.7213	3.6477	3.6544	-0.0758	yes
500	50	3.7124	3.7012	3.6163	3.6272	-0.0904	yes
500	100	3.6624	3.6524	3.5642	3.5752	-0.0932	yes
500	250	3.5774	3.5751	3.4849	3.4915	-0.0914	yes
500	500	3.4852	3.4888	3.4080	3.4038	-0.0790	yes
1000	25	3.4772	3.4802	3.4191	3.4224	-0.0596	yes
1000	50	3.4719	3.4735	3.3945	3.4031	-0.0782	yes
1000	100	3.4591	3.4605	3.3740	3.3848	-0.0858	yes
1000	250	3.4232	3.4229	3.3369	3.3450	-0.0861	yes
1000	500	3.3790	3.3813	3.2930	3.3016	-0.0872	yes

- **26 of 30 cells satisfy B ≤ 0.01.** The four that fail are (x, y) = (0, 250), (0, 500), (50, 500) and (100, 500), with B = 0.226, 0.622, 0.139 and 0.028. Cell (50, 250) passes narrowly (B = 0.006).
- **Averaging mostly helps.** Most B values are negative, down to −0.108 at (50, 50): the average is better than both endpoints, not just no worse.
- **Where it fails:** runs that share none or little of the prefix and then train a long time apart. With x = 0 the barrier is negative for y ≤ 100 and jumps to 0.226 at y = 250 and 0.622 at y = 500 (the two runs share only their initialization). The failure region shrinks as x grows: it ends at y = 500 for x = 50 and 100, and no cell with x ≥ 250 fails. It looks governed by the ratio y/x, roughly y ≥ 5x.
- **Where it works best:** short branches (y ≤ 100) from any x ≥ 50, and every cell with x ≥ 250 (B between −0.055 and −0.108).
- **Weight average versus ensemble:** the averaged weights beat the ensemble in 16 of 30 cells, mostly the long-prefix, short-branch ones (x ≥ 250 with y ≤ 250, and x = 1000 throughout).

(c) Interpolation curves



x	y	α = 0.0	α = 0.25	α = 0.5	α = 0.75	α = 1.0	ensemble
0	100	5.0295	4.9980	5.0148	5.0005	5.0319	4.9213
0	500	3.7489	3.9020	4.3647	3.8835	3.7371	3.6398
50	250	4.1274	4.1105	4.1418	4.1211	4.1450	4.0308
100	500	3.6636	3.6553	3.6875	3.6507	3.6548	3.5624
250	500	3.5736	3.5247	3.5163	3.5215	3.5698	3.4820
1000	500	3.3790	3.3127	3.2930	3.3133	3.3813	3.3016

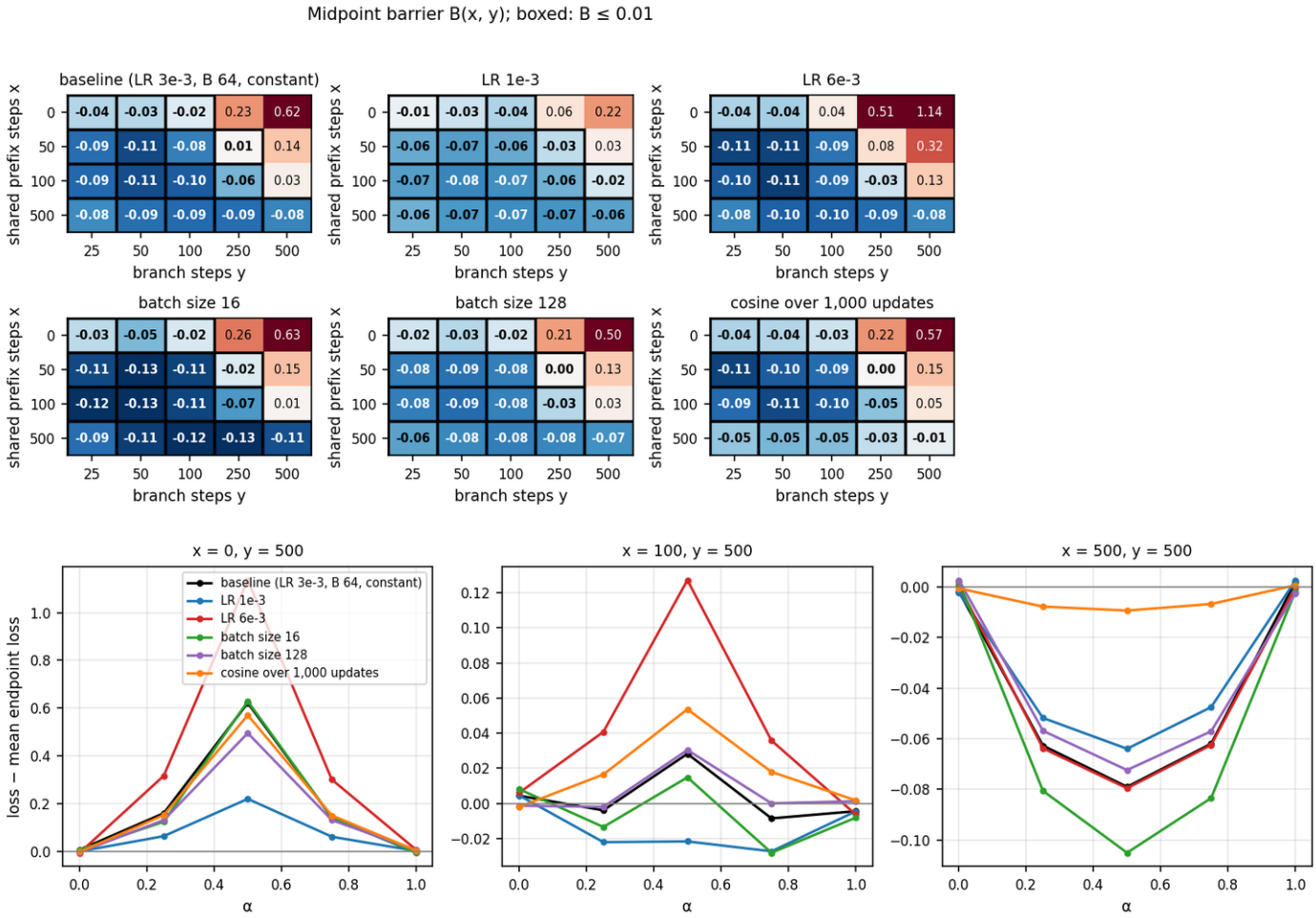
- **x = 0, y = 500 (B = +0.622):** a clear barrier. The loss rises from 3.75 at the endpoints to 4.36 at α = 0.5, so the two runs are in different basins. Their ensemble (3.64) is still better than either endpoint.
- **x = 0, y = 100 (B = −0.016):** after only 100 steps from the same initialization the line between the runs is flat or slightly better than the endpoints.
- **x = 50, y = 250 and x = 100, y = 500 (B = +0.006, +0.028):** near the boundary the curve is almost flat with a small bump at α = 0.5.
- **x = 250, y = 500 and x = 1000, y = 500 (B = −0.055, −0.087):** a clear dip with the minimum at α = 0.5 and a roughly quadratic shape, as

expected inside one basin. At x = 1000 the midpoint (3.293) is below the ensemble (3.302).

- **Midpoint versus ensemble:** the ensemble is better than the midpoint where there is a barrier or the curve is flat (x = 0 and the small-x cells), and the midpoint is better where the curve dips (x ≥ 250 with short branches, and x = 1000). Averaging weights cannot beat the ensemble inside the failing cells, but in a shared basin it can match or exceed it at one forward pass instead of two. The curve's minimum is not always at α = 0.5: at (0, 100) and (50, 250) the α = 0.25 and 0.75 points are lower than the midpoint.

(d) What controls whether runs share a basin

One knob changed at a time from the baseline (LR 3e-3, batch 64, constant LR after warmup), on a smaller grid x ∈ {0, 50, 100, 500}, y ∈ {25, 50, 100, 250, 500} (two branches per cell, 40 extra branch runs). Batch-size variants keep the 94-update warmup of the baseline; the cosine variant decays to zero at update 1,000, so the branches from x = 500 are fully annealed.



Variant	B(0, 500)	B(100, 500)	B(500, 500)	cells with B > 0.01	most negative B	ensemble gain (x = 500, y = 500)
baseline (LR 3e-3, B 64, constant)	+0.622	+0.028	-0.079	4 [(0, 250), (0, 500), (50, 500), (100, 500)]	-0.108	0.083
LR 1e-3	+0.219	-0.022	-0.064	3 [(0, 250), (0, 500), (50, 500)]	-0.078	0.071
LR 6e-3	+1.137	+0.127	-0.080	6 [(0, 100), (0, 250), (0, 500), (50, 250), (50, 500), (100, 500)]	-0.114	0.087
batch size 16	+0.628	+0.015	-0.105	4 [(0, 250), (0, 500), (50, 500), (100, 500)]	-0.132	0.109
batch size 128	+0.495	+0.030	-0.072	4 [(0, 250), (0, 500), (50, 500), (100, 500)]	-0.092	0.079
cosine over 1,000 updates	+0.570	+0.054	-0.009	4 [(0, 250), (0, 500), (50, 500), (100, 500)]	-0.111	0.018

- **Learning rate is the strongest control.** The barrier at (0, 500) is 0.22 at LR 1e-3, 0.62 at 3e-3 and 1.14 at 6e-3, and the region that fails the B ≤ 0.01 test grows with LR: 3 cells at 1e-3 (all with x ≤ 50), 4 at 3e-3, 6 at 6e-3, including (0, 100) and (50, 250), which pass at the baseline. At 1e-3 even the (100, 500) cell passes (−0.022). A larger LR lets two runs drift further apart in the same time, so they need a longer shared prefix to stay in one basin.
- **Batch size barely changes where averaging fails.** The failing cells are exactly the baseline's at batch 16 and 128 and the barrier at (0, 500) moves only from 0.63 (B = 16) to 0.62 (64) to 0.50 (128). Batch size does change how much averaging helps inside the basin: the most negative barrier is −0.13 at batch 16 and −0.09 at 128, and the ensemble gain at (500, 500) is 0.109 against 0.079, so the noisier small-batch iterates have more to average out.
- **Annealing removes the benefit of averaging, not the basin.** With the cosine schedule the x = 500 cells have decayed LR, and B(500, 500) is −0.009 with an ensemble gain of 0.018, against −0.079 and 0.083 at constant LR. The runs agree (no barrier), but there is almost no oscillation left to average away. The cells with x ≤ 100, where the LR is still high, behave like the baseline (the same 4 cells fail).
- **Pattern.** Whether two runs are averageable is set by how far noise at the current LR moves them apart (large LR, long branch) against how much of the trajectory they share (large x). Their gain from averaging, when it works, is set by the oscillation amplitude: larger with high LR and small batches, near zero once the LR has decayed. This matches the river-valley picture: averaging cancels the motion across the valley, and it only fails

when the runs have chosen different valleys.

- **Experiments that would test this.** (i) Decay both branches of a failing cell to zero LR before averaging, to see whether the barrier is oscillation across a shared valley or a different basin. (ii) A finer LR grid at x = 50 and 100, to check whether the failure boundary moves as y ∝ x/LR. (iii) More branch pairs per cell, since each cell here has a single pair of branches and no estimate of the seed-to-seed spread of B. (iv) Vary the data-order seed against the initialization seed at x = 0, to separate the two sources of divergence.

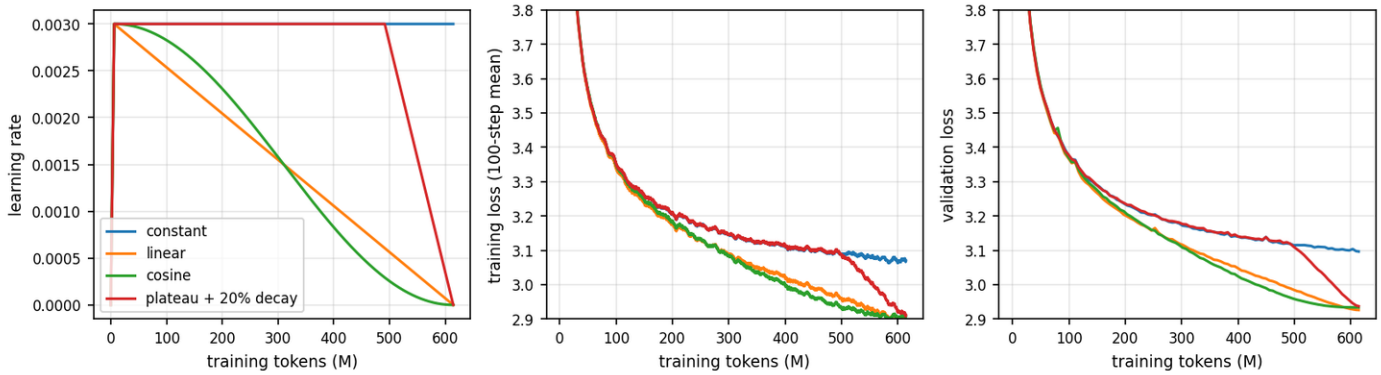
Synthesis

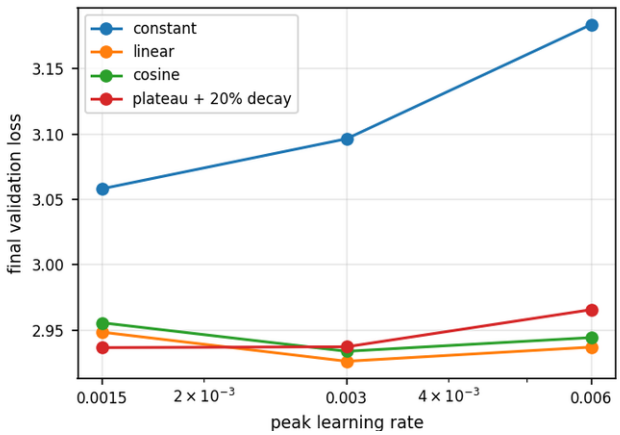
Two runs forked from the same checkpoint behave like one run for the loss at the endpoints (it depends only on x + y), and a 0.5/0.5 prediction ensemble always improves on them by 0.05–0.11. Averaging weights works ($B \leq 0.01$) in 26 of the 30 baseline cells and usually beats both endpoints, with the average better than the ensemble when the shared prefix is long. It fails only when two runs have trained apart for a long time from a short prefix (no shared steps with $y \geq 250$, or $y \geq 5x$), where the barrier reaches 0.6. Raising the LR extends that failure region and a longer shared prefix shrinks it, while batch size does not move it and an annealed LR removes the gain from averaging. The pattern is consistent with a basin that is established early and a river-valley oscillation that averaging cancels.

Problem 2: River-valley dynamics

Default d8, 614M tokens, AdamW. Regret R_M(t) = L(θ_t^M) − L(θ_t^ref), compared at equal token counts. EMA decays tracked during every run (evaluation only): 0.9, 0.99, 0.999, 0.9995 (timescales of about 10, 100, 1,000 and 2,000 updates of the 9,375).

(a) Schedules





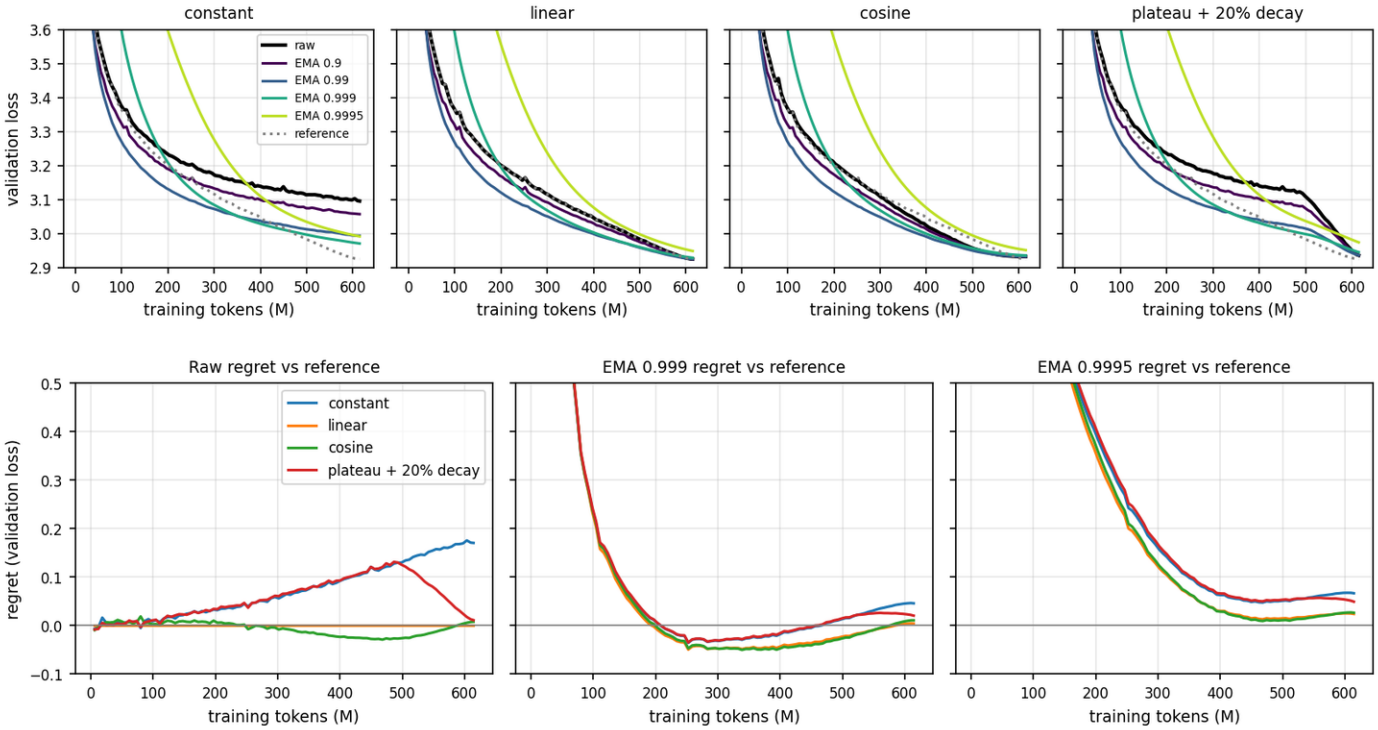
- **Constant after warmup** keeps decreasing slowly but is 0.17 above the best run at the end (3.097 vs 2.926). Its gap to the decayed runs opens steadily from about 100M tokens, as the decayed schedules' LRs fall.
- **Linear and cosine** are almost identical until about 150M tokens, then cosine is slightly lower (by up to 0.03 around 400M tokens) while its LR is lower; they converge near the end and linear finishes ahead (2.926 vs 2.934).
- **Plateau + 20% decay** tracks the constant curve until the decay starts at about 490M tokens (80%), then drops about 0.18 in the last 120M tokens and ends at 2.937, 0.011 above linear. The drop is almost entirely made in the decay phase, as the river-valley picture predicts: the high-LR plateau hides progress that annealing reveals.
- **Peak LR.** 3e-3 is best for linear (2.926) and cosine (2.934). For plateau + decay 1.5e-3 and 3e-3 tie (2.9365 vs 2.9371), and 6e-3 is worse (2.966). Constant prefers the lower LR (3.058 at 1.5e-3, 3.097 at 3e-3, 3.184 at 6e-3): without annealing a larger LR leaves more oscillation.
- **Reference for (b):** linear at 3e-3, the lowest final validation loss (2.9261).

Reference: linear lr=0.003, final 2.9261

Schedule	Peak LR	Final val loss	EMA 0.9	EMA 0.99	EMA 0.999	EMA 0.9995	Raw regret	Best-EMA regret
constant	0.0015	3.0582	3.0305	2.9804	2.9617	2.9829	+0.1321	+0.0356
constant	0.003	3.0965	3.0580	2.9948	2.9719	2.9926	+0.1704	+0.0458
constant	0.006	3.1839	3.1311	3.0529	3.0303	3.0547	+0.2579	+0.1042

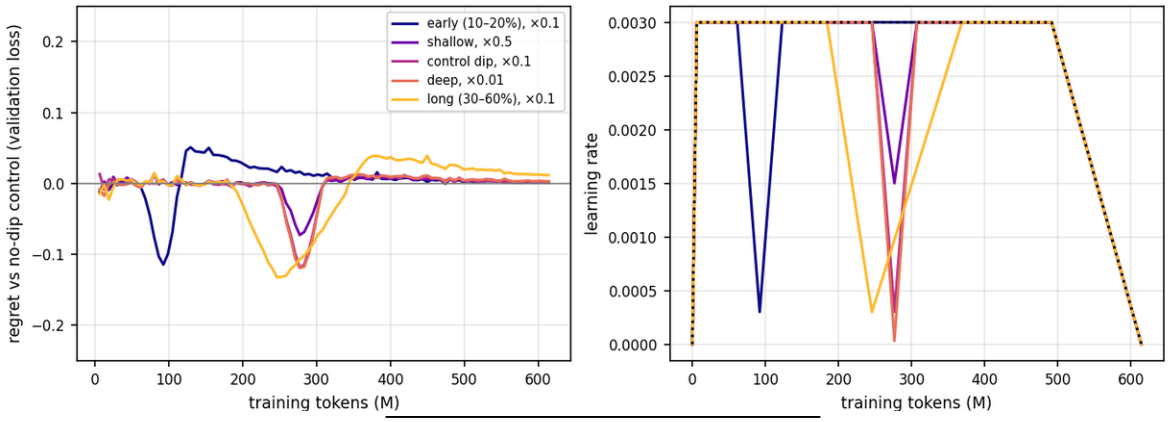
Schedule	Peak LR	Final val loss	EMA 0.9	EMA 0.99	EMA 0.999	EMA 0.9995	Raw regret	Best-EMA regret
linear	0.0015	2.9483	2.9484	2.9485	2.9523	2.9685	+0.0223	+0.0223
linear	0.003	2.9261	2.9261	2.9263	2.9301	2.9499	+0.0000	+0.0000
linear	0.006	2.9369	2.9369	2.9371	2.9431	2.9747	+0.0108	+0.0108
cosine	0.0015	2.9555	2.9555	2.9556	2.9583	2.9707	+0.0295	+0.0295
cosine	0.003	2.9337	2.9337	2.9337	2.9368	2.9524	+0.0076	+0.0076
cosine	0.006	2.9442	2.9442	2.9442	2.9486	2.9741	+0.0181	+0.0181
plateau + 20% decay	0.0015	2.9365	2.9366	2.9368	2.9433	2.9676	+0.0105	+0.0105
plateau + 20% decay	0.003	2.9371	2.9372	2.9374	2.9464	2.9753	+0.0111	+0.0111
plateau + 20% decay	0.006	2.9656	2.9657	2.9656	2.9839	3.0250	+0.0395	+0.0395

(b) EMA and regret



- **EMA helps a lot without a decay, and not at all with one.** For constant LR 3e-3 the final loss falls from 3.097 (raw) to 2.972 with EMA 0.999, which removes 0.125 of the 0.170 gap to the reference. At 1.5e-3 it falls from 3.058 to 2.962. For linear, cosine and plateau + decay the raw weights are already at the valley floor, and any EMA is equal or worse at the end (EMA 0.999 is 0.003 to 0.009 worse, EMA 0.9995 is 0.02 to 0.04 worse) because it averages in older, higher-loss iterates.
- **Timescale.** 0.999 (about 1,000 updates) is best for constant LR. 0.9 and 0.99 are too short to average out the oscillation (3.058 and 2.995 at 3e-3), and 0.9995 is too long: it lags the downward trend. The EMA is not bias-corrected, so the 0.999 and 0.9995 curves are far above the raw loss for the first 200M and 450M tokens (they still contain the initialization), and the regret there is meaningless.
- **Mid-training.** Between about 200M and 450M tokens the EMA 0.999 of every schedule is below the reference's raw loss (regret down to -0.05): an EMA of a high-LR run estimates the annealed loss without annealing.
- **What gap remains.** After EMA the constant run is still 0.046 above the reference (27% of the original gap), and EMA 0.9995 does worse (0.067). Averaging closes about three quarters of the gap between schedules, not all of it. The gap between the decayed schedules themselves (0.008–0.011) is unchanged by averaging.

(c) Dip and rewarmup

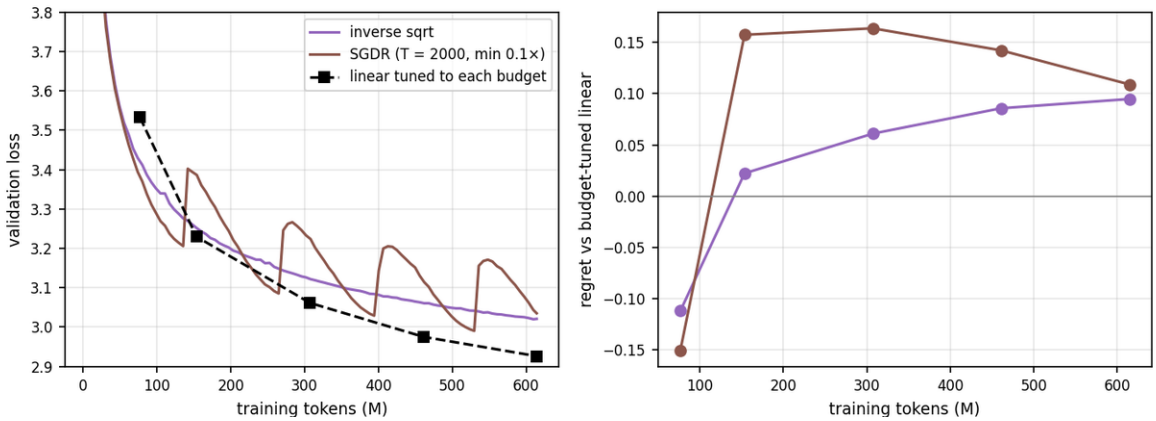


Dip	min regret (tokens M)	final regret
early (10–20%), ×0.1	-0.1148 (92)	+0.0012
shallow, ×0.5	-0.0731 (277)	+0.0010
control dip, ×0.1	-0.1189 (277)	+0.0015
deep, ×0.01	-0.1198 (277)	+0.0030
long (30–60%), ×0.1	-0.1328 (246)	+0.0116

Control: plateau + 20% decay at 3e-3 with no dip, all runs sharing the same final cooldown. The regret is measured against the control's raw validation loss at the same token count.

- **The improvement appears at once and does not survive rewarmup.** Each dip lowers the validation loss by 0.07 to 0.13 within about 50M tokens, and the gain disappears as the LR returns to 3e-3. After the final cooldown every run ends within 0.003 of the control, except the long dip (+0.012).
- **Depth.** The size of the improvement saturates with depth: ×0.5 gives −0.073, ×0.1 gives −0.119 and ×0.01 gives −0.120. Going from ×0.1 to ×0.01 adds nothing, consistent with reaching the valley floor once the oscillation is small enough.
- **Duration.** The longer 30–60% dip gives the largest improvement (−0.133) but recovers more slowly, ending at +0.012 because 30% of training was spent at a lower LR than the control.
- **Timing.** The early dip at 10–20% gives −0.115 at its bottom and then overshoots: after rewarming the loss is +0.05 above the control, decaying to +0.001 over the rest of training.
- **Predictability.** The depth and duration of the regret follow the LR profile: the loss tracks the instantaneous learning rate, dropping when it dips and returning when it rewarms, and the end result is fixed by the final cooldown rather than the earlier history. The final regret after a short dip is within seed-level noise.

(d) Horizon-free schedules



Budget (tokens)	linear tuned	invsqrt	regret	SGDR	regret
77M	3.5332	3.4214	-0.1118	3.3830	-0.1502
154M	3.2296	3.2518	+0.0222	3.3868	+0.1572
307M	3.0613	3.1223	+0.0609	3.2249	+0.1636
461M	2.9752	3.0609	+0.0857	3.1173	+0.1420
614M	2.9261	3.0207	+0.0947	3.0350	+0.1089

Warmup is fixed at 94 updates for both horizon-free runs. The budget-tuned linear runs use the default 1% warmup, which is only 11 updates at the 77M budget; that run is probably handicapped and the −0.11 and −0.15 entries at that budget should not be trusted.

- **Inverse square root** has regret that grows with budget: +0.022 at 154M, +0.061 at 307M, +0.086 at 461M and +0.095 at 614M. It cannot reach

the loss that a budget-tuned decay to zero does, and the loss falls more slowly than the optimal anneal.

- **SGDR** has a sawtooth: each restart raises the loss by about 0.15 and it then decays again over the 2,000-update cycle. Measured at the budgets, its regret is +0.11 to +0.16 because those budgets fall mid-cycle. At the end of a cycle (low LR) it is close to the tuned curve: 3.085 at 265M and 3.029 at 394M against 3.11 and 3.01 for tuned linear (interpolated between the measured budgets), and 2.990 at 530M against 2.953.
- **Verdict.** If the budget is unknown, neither is a replacement for annealing: both stay 0.02–0.1 above a tuned schedule once the budget exceeds about 150M tokens. SGDR is a useful alternative only if you can stop at a cycle end, where it comes within 0.02–0.04 of tuned linear. Inverse square root is the more predictable of the two.

Synthesis

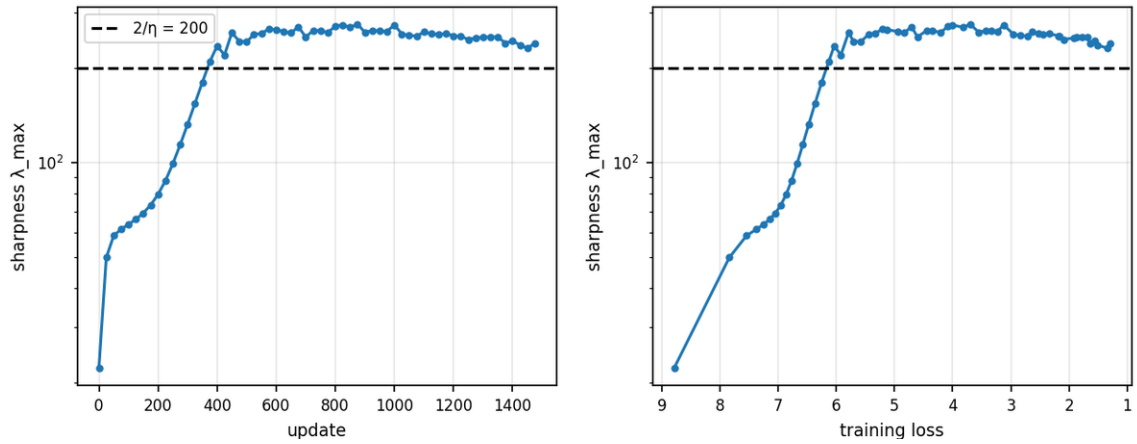
The river-valley picture fits our runs. A constant LR leaves the model 0.17 above what annealing reaches, and almost all of the gap is closed in the last 20% of training when the LR decays (plateau + decay) or by averaging the iterates (EMA 0.999, three quarters of the gap), which both estimate the position at the bottom of the valley. A temporary dip shows the same thing from the other side: the loss falls as soon as the LR falls and rises when it is restored, and only the final cooldown determines the result, so dips are free but also useless. Schedules that decay to zero reach similar final losses (2.926–2.937), with linear best and cosine slightly ahead mid-training, and the best peak LR (about 3e-3) is flat for them. Horizon-free schedules give up 0.02–0.1 in loss for not knowing the budget, with SGDR competitive only at cycle ends.

Problem 3: The edge of stability

Thresholds: SGD $\eta\cdot\lambda_max \to 2$; AdamW with $\beta1 = 0.9$ and a frozen preconditioner, $\eta\cdot\lambda_max^{pre} \to 2(1+\beta1)/(1-\beta1) = 38$ (and 2, 18 and 78 for $\beta1 = 0$, 0.8 and 0.95). λ_max is measured on a fixed probe: the 32 training sequences themselves in part (a), 64 held-out sequences in parts (b) and (c).

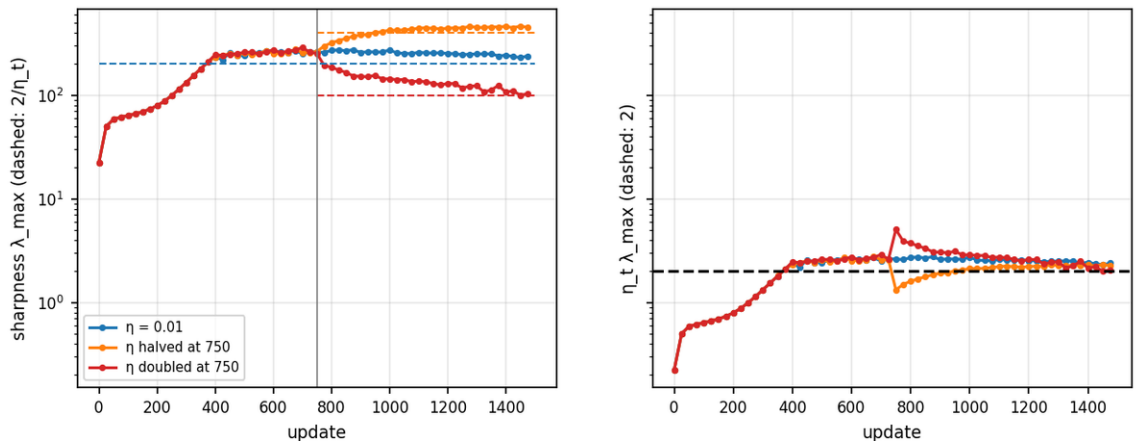
(a) Full-batch replication

i. The clean case ($N = 32$, $\eta = 0.01$, 1,500 updates, 16× embedding initialization)



- **Progressive sharpening.** λ_max starts at 22 ($\eta\cdot\lambda_max = 0.22$) and climbs steadily as the loss falls from 8.8 to 6, crossing $2/\eta = 200$ at update 375.
- **Hovering at the edge.** After the crossing λ_max stays at 250–270 for the rest of the run (median $\eta\cdot\lambda_max = 2.56$ from 100 updates after the crossing), a little above $2/\eta$, and then drifts down slowly to about 240 while the loss goes from 6 to 1.3.
- **The loss stops decreasing monotonically.** The training loss never rises before the crossing (0% of updates) and rises on 40% of updates after it, while the long-run trend is still downward.
- **Match with the predictions.** This is the pattern in Cohen et al.: sharpening up to $2/\eta$, then hovering there with non-monotone loss. The one difference is the level: it hovers at about 1.25–1.3 × $2/\eta$ rather than just at it, and never falls back under $2/\eta$, so it does not leave the edge.

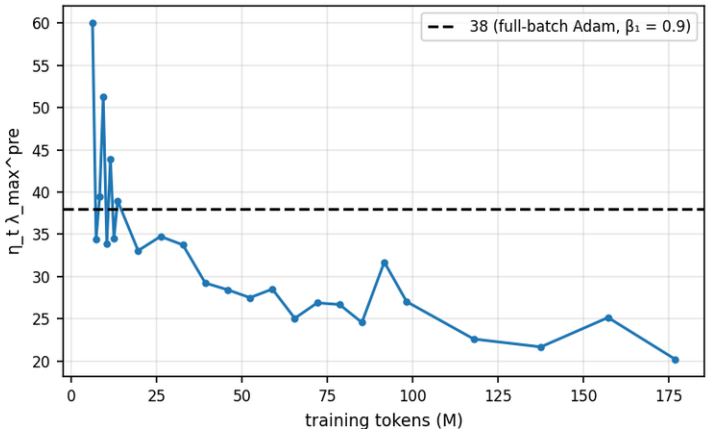
ii. Changing η mid-run



Run	λ_max, updates 600–750	λ_max, updates 1100–1500	η_t·λ_max after	Threshold 2/η_t
η = 0.01	262	250	2.50	200
η halved at 750	258	449	2.25	400
η doubled at 750	266	123	2.46	100

- **Expected from the theory:** λ_max follows 2/η_t, up to about 400 after halving and down to about 100 after doubling.
- **Both runs move to the new edge.** λ_max rises 1.7× after halving and falls to 0.46× after doubling, so η_t·λ_max ends at 2.25 and 2.46, close to the 2.5 of the clean case.
- **The response is not symmetric.** Halving η drops η·λ_max to 1.3, below the edge, and sharpness has to rise again by progressive sharpening, which takes about 250 updates (λ_max reaches about 420 around update 1000). Doubling η pushes η·λ_max to 5, above the edge, and λ_max falls within about 100 updates (to about 150 by update 850) through self-stabilization, then keeps creeping down to 123. Reducing sharpness is much faster than raising it.

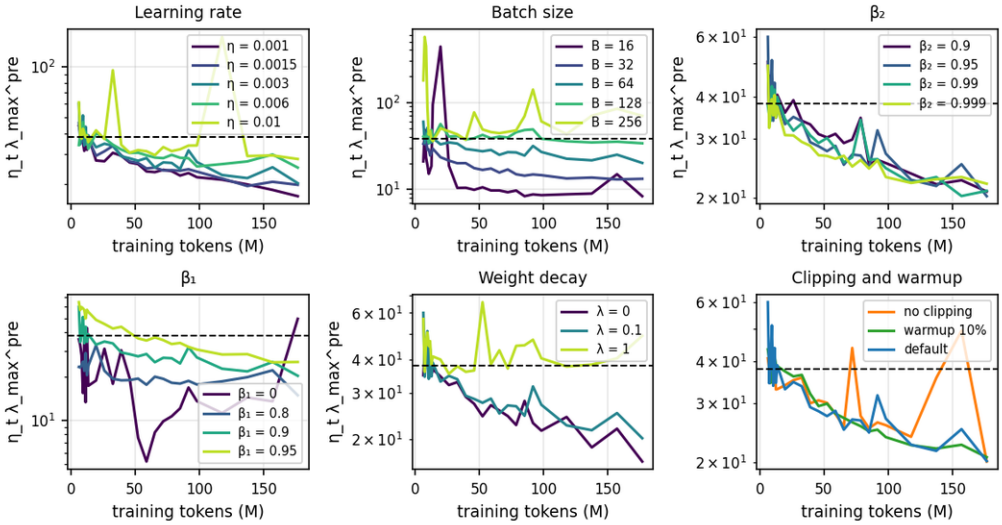
(b) Minibatch AdamW



Base recipe: η = 3e-3, batch 64, constant LR after warmup, 0.32 epochs (197M tokens).

- **It does not stay at 38.** After warmup the quantity starts at 35–60, dips under 38 by about 20M tokens, and then drifts steadily down to about 20 by 175M tokens.
- **So no: minibatch AdamW does not train at the edge** of the full-batch Adam threshold. At 100–190M tokens it sits at 22 (58% of 38) and is still falling. The early values above 38 are the transient right after warmup.

(c) Hyperparameter exploration



Median λ_max^pre and η·λ_max^pre over 100–190M tokens (about 60% of the run), one change at a time from the base recipe (η = 3e-3, B = 64, β1 = 0.9, β2 = 0.95, λ = 0.1). λ_max^pre is the top eigenvalue of D^(-1/2) H D^(-1/2); it is in the thousands because the preconditioner divides by a small $\sqrt{\hat{v}}$, so it is only meaningful relative to 1/η:

Change	Value	λ_max^pre	η·λ_max^pre	Final train loss
Base		7,382	22.1	3.235
Learning rate	1e-3	19,160	19.2	3.246
	1.5e-3	13,462	20.2	3.231
	6e-3	4,460	26.8	3.305
	1e-2	2,946	29.5	3.378

Change	Value	λ_max^pre	η·λ_max^pre	Final train loss
Batch size	16	2,972	8.9	3.281
	32	4,454	13.4	3.295
	128	11,704	35.1	3.201
	256	23,449	70.3	3.329
β2	0.9	7,414	22.2	3.240
	0.99	7,193	21.6	3.238
	0.999	7,474	22.4	3.255
β1	0	4,645	13.9	3.405
	0.8	6,387	19.2	3.246
	0.95	8,910	26.7	3.255
Weight decay	0	6,773	20.3	3.200
	1.0	13,129	39.4	3.507
Gradient clipping	off	9,935	29.8	3.251
Warmup	10% (vs 3.125%)	7,430	22.3	3.241

- **Batch size matters most, and in the direction the noise picture predicts.** η·λ_max^pre grows from 8.9 at B = 16 to 70 at B = 256 (about B^0.75), so larger batches sit at or above 38 and small batches well below it. Smaller batches add gradient noise that keeps the model away from the edge; large batches approach the full-batch behaviour (and the early transient of 100+ for B = 256 and 16 shows how noisy the measurement is).
- **Learning rate has a weak, monotone effect.** A 10× larger LR raises η·λ_max^pre only 1.5× (19 to 29.5): sharpness falls as η rises (λ_max^pre scales roughly like η^-0.8), so the product moves little. The quantity is closer to constant than λ_max itself, which supports η·λ_max as the natural variable.
- **β2 has no effect** (21.6–22.4 over a 1,000× range in $1-\beta2$), and so does warmup length.
- **β1 moves it up, but far less than the threshold.** The measured value grows 13.9, 19.2, 22.1, 26.7 for β1 = 0, 0.8, 0.9, 0.95, while $2(1+\beta1)/(1-\beta1)$ grows 2, 18, 38, 78. The ratio to the threshold goes 6.9, 1.07, 0.58, 0.34: at β1 = 0 the run is far above the quadratic threshold of 2 yet trains stably (the preconditioner and gradients are not frozen), and with large β1 it sits well below. Momentum does raise the stable sharpness, as in the lecture's full-batch picture, but not by the full factor in the minibatch regime.
- **Weight decay and clipping.** λ = 1 gives 39 but a much worse loss (3.507, so a different regime), and removing the gradient clip raises it to 29.8.
- **What decides whether the run is at the edge.** The tendency to be near the threshold goes up with batch size, β1 and LR, and goes down with noise (small batches). Even where it does reach 38 (B ≥ 128, λ = 1) it does not hold there.

Synthesis

Full-batch training on a tiny set reproduces the edge of stability: sharpness rises until η·λ_max reaches about 2.5, and then holds there, with the loss still falling but no longer monotonically. When η is changed the sharpness moves to the new 2/η, quickly when it must go down and slowly when it can go up. The minibatch AdamW setting we actually use is different: η·λ_max^pre starts near 38 after warmup, then falls to about 20 and keeps falling, well below the full-batch threshold. Noise appears to keep the model off the edge: smaller batches sit far below it (9 at B = 16), larger batches approach it (70 at B = 256), and the measured level changes only weakly with LR and β1 and not at all with β2, so the full-batch threshold is a useful upper reference and not a good prediction for minibatch training.

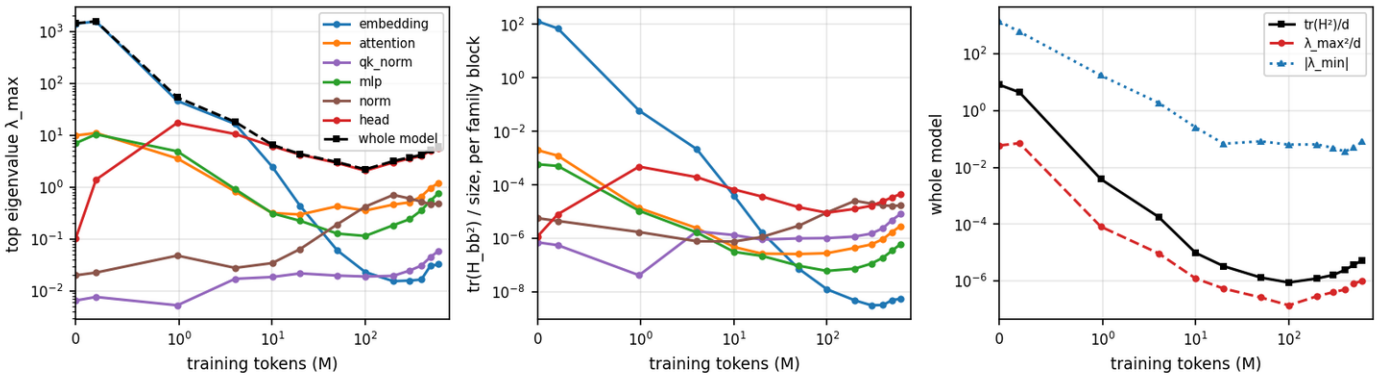
Problem 4: Inside the Hessian

Hessian of the mean next-token loss on the 64 held-out probe sequences, FP32, TF32 off.

(a) What the Hessian looks like

Default d8 AdamW run, 13 checkpoints from initialization to 614M tokens; all measurements on the 64 held-out probe sequences in FP32. tr(H²)/d is estimated by Hutchinson block energies (4 probes), the family λ_max values by Lanczos on each principal block.

Which family is sharpest, and how it changes



tokens	λ_max	λ_min	tr(H²)/d	sharpest family (λ_max)	λ_max² / tr(H²)
0.0M	1431.503	-1410.061	8.18	embedding (1414.298)	0.007
0.2M	1571.742	-601.660	4.43	embedding (1553.920)	0.016
1.0M	53.364	-17.453	0.00388	embedding (45.811)	0.021
4.0M	18.055	-1.896	0.000181	embedding (16.491)	0.052
10.0M	6.502	-0.256	9.74e-06	head (6.139)	0.125
20.0M	4.329	-0.069	3.33e-06	head (4.213)	0.162
50.0M	3.024	-0.083	1.33e-06	head (2.911)	0.199
100.0M	2.195	-0.064	8.86e-07	head (2.067)	0.157

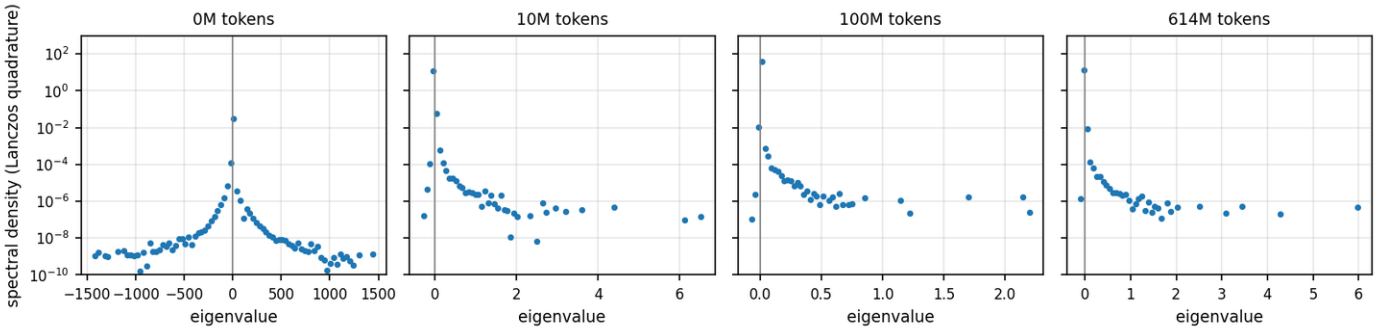
tokens	λ_max	λ_min	tr(H²)/d	sharpest family (λ_max)	λ_max² / tr(H²)
200.0M	3.164	-0.065	1.23e-06	head (2.980)	0.236
300.0M	3.742	-0.049	1.62e-06	head (3.558)	0.249
400.0M	4.155	-0.036	2.44e-06	head (3.963)	0.205
500.0M	5.282	-0.050	3.7e-06	head (5.021)	0.218
614.4M	5.967	-0.080	5.36e-06	head (5.642)	0.192

Top eigenvalue of each family's own block:

tokens	embedding	attention	qk_norm	mlp	norm	head
0.0M	1414.298	9.879	0.007	7.011	0.020	0.102
0.2M	1553.920	11.032	0.008	10.397	0.022	1.389
1.0M	45.811	3.563	0.005	4.881	0.048	17.367
4.0M	16.491	0.821	0.017	0.911	0.028	10.538
10.0M	2.457	0.316	0.018	0.312	0.034	6.139
20.0M	0.432	0.296	0.022	0.224	0.064	4.213
50.0M	0.060	0.427	0.020	0.127	0.191	2.911
100.0M	0.023	0.352	0.019	0.114	0.420	2.067
200.0M	0.015	0.460	0.019	0.184	0.706	2.980
300.0M	0.016	0.508	0.025	0.241	0.602	3.558
400.0M	0.016	0.658	0.031	0.355	0.523	3.963
500.0M	0.030	0.959	0.044	0.543	0.471	5.021
614.4M	0.033	1.208	0.059	0.764	0.475	5.642

- **Early training is dominated by the input embedding, then the head takes over.** At initialization the sharpest block is the embedding (λ_max 1,414, whole-model 1,432), up to 4M tokens. From 10M tokens on, the output head is the sharpest family at every checkpoint, by a margin of 2.5–10× over the next family, and the whole-model λ_max is within 3–6% of the head's.
- **Two phases.** λ_max falls by more than 200× in the first 20M tokens (1,432 to 4.3) and keeps falling to 2.2 at 100M, then rises to 6.0 at 614M. tr(H²)/d follows the same U shape (8.2 at initialization, 8.9e-7 at 100M, 5.4e-6 at 614M).
- **Families differ in how they move.** The embedding block collapses (1,414 to 0.033), the head block first rises (0.10 at initialization, 17 at 1M) then falls and rises again (2.07 at 100M, 5.64 at the end), the attention and MLP blocks fall to a minimum at 20–100M tokens and then rise again (to 1.21 and 0.76), and the norm gains rise from 0.02 to 0.7 at 200M and settle at about 0.48. The QK-norm gains stay small (0.007–0.06).
- **A few directions carry a growing share of the curvature.** λ_max² / tr(H²), the share of ‖H‖_F² in the top eigenvector, goes from 0.7% at initialization to about 20% at the end.

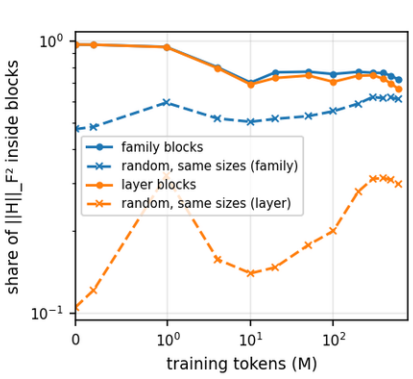
The spectrum

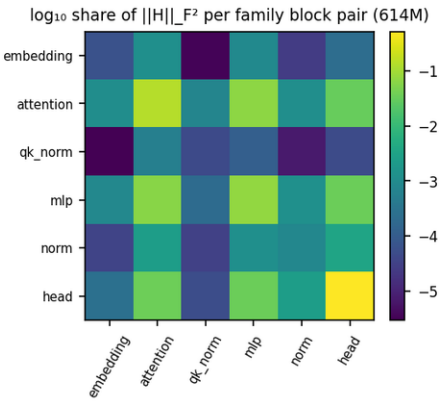


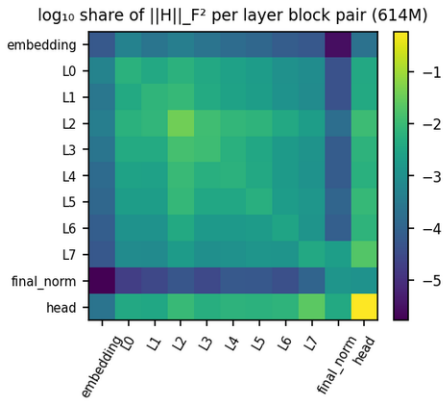
tokens	λ_min	λ_max	mass at λ < −0.01	mass within ±0.01 of 0	99.99% quantile	99.9999% quantile	top-3 eigenvalues
0M	-1410.061	1431.503	4.51e-03	0.0000	58.556	684.375	1431.503, 1307.342, 1250.305
10M	-0.256	6.502	4.31e-04	0.9983	0.069	1.212	6.502, 6.095, 4.398
100M	-0.064	2.195	2.70e-05	0.9998	0.013	0.305	2.195, 2.156, 1.692
614M	-0.080	5.967	4.96e-05	0.9993	0.023	0.849	5.967, 4.289, 3.429

- **The bulk is a spike at zero.** 99.8–99.98% of the spectral mass lies within $|\lambda| \le 0.01$ from 10M tokens onward, with a long, thin tail of positive eigenvalues out to a few. At initialization it is much wider and nearly symmetric ($|\lambda|$ up to 1,400).
- **There are outliers.** A handful of eigenvalues separate from the continuous part: at 614M the top eigenvalues are 5.97, 4.29, 3.43, against a 99.9999% spectral quantile of 0.85. The outliers are isolated spikes in the density, not a continuous tail.
- **There are negative eigenvalues, but they are small.** The most negative is −0.08 at 614M tokens (about 1.3% of λ_max), −0.26 at 10M and −0.06 at 100M, and the mass below −0.01 is only 3e-5 to 4e-4. At initialization it is large (−1,410) and symmetric with λ_max. So training mostly sits in a region with almost no negative curvature, which is the "saddles are not a problem" picture of lecture 1.
- **Implications for training.** Almost all directions are flat ($|\lambda| < 0.01$), so a step in them changes the loss very little, and the loss is controlled by a few sharp directions: the learning rate is limited by the top few eigenvalues (about 6 at the end), while progress comes from the large flat bulk.

Block structure



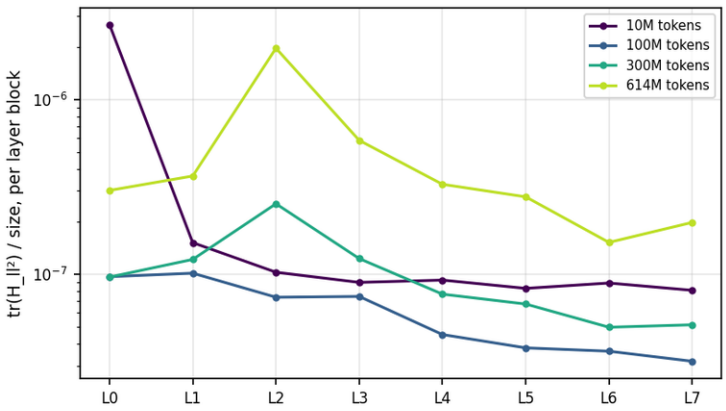




tokens	family blocks: share inside	layer blocks: share inside	random family	random layer
0.0M	0.968	0.969	0.474	0.105
0.2M	0.967	0.968	0.483	0.121
1.0M	0.950	0.948	0.592	0.319
4.0M	0.799	0.794	0.518	0.157
10.0M	0.702	0.691	0.504	0.140
20.0M	0.766	0.730	0.517	0.147
50.0M	0.770	0.745	0.529	0.178
100.0M	0.755	0.707	0.551	0.200
200.0M	0.769	0.744	0.588	0.279
300.0M	0.763	0.747	0.622	0.311
400.0M	0.762	0.726	0.615	0.313
500.0M	0.744	0.695	0.618	0.309
614.4M	0.719	0.665	0.610	0.297

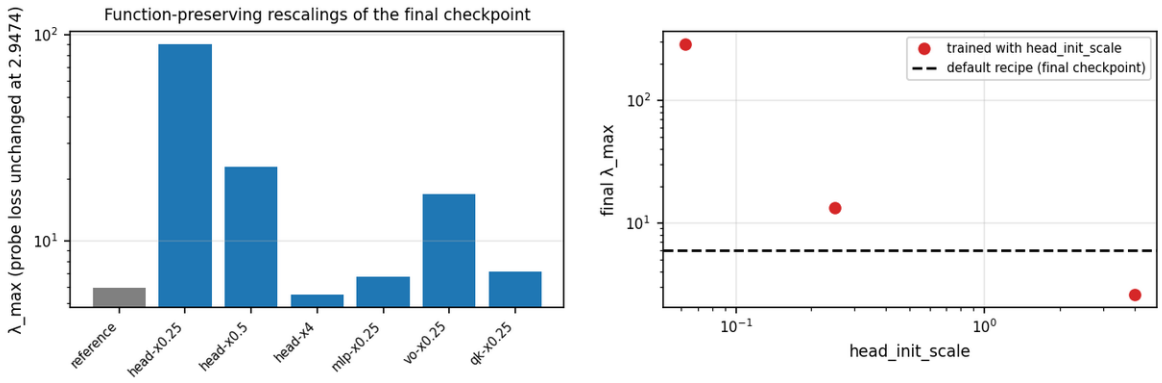
Block energies are Hutchinson estimates of ‖H_cb‖_F² with 4 probes per block. ”Random” assigns each parameter to a block of the same size as the real family or layer block uniformly at random. At 614M tokens the four probes give 0.54–0.65 for the random family partition and 0.28–0.31 for the random layer partition.

- **The Hessian is close to block-diagonal at initialization and less so later.** The share of ‖H‖_F² inside family blocks is 0.97 at initialization and falls to about 0.70 by 10M tokens, then stays at 0.72–0.77; for layer blocks it falls from 0.97 to 0.69 at 10M and then to 0.67 at the end.
- **Against a random partition of the same sizes:** the layer blocks are far more block-diagonal than chance, 0.67 against 0.30 at 614M (2.2×; and 0.97 against 0.10 at initialization). The family blocks are only moderately above their random baseline (0.72 against 0.61 at 614M, with a spread of about ±0.05), because the MLP block holds 64% of all parameters and any partition with such a big block keeps a lot of energy inside it.
- **Where the energy is.** At 614M the head–head block alone holds 52% of ‖H‖_F² (63% at 100M), attention–attention 13% and MLP–MLP 7%, and the attention–MLP cross blocks another 13%. The head's coupling to the other families is 14%.
- **Cross-layer structure.** The layer-by-layer heat map shows that within-layer blocks dominate, with extra coupling between neighbouring middle layers and between the last layers and the head.



- **Curvature does not grow with depth.** At 614M tokens the per-layer curvature tr(H_ll²)/size is 3.0e-7 (L0), 3.7e-7 (L1), 2.0e-6 (L2, the peak), 5.8e-7 (L3), and then falls to 2.0e-7 at L7. At 10M tokens it is largest in the first layer (2.7e-6 at L0) and decreases with depth, and at 100M it decreases from 1.0e-7 (L1) to 3.2e-8 (L7). It is largest in the early layers and lower in the last ones.

(b) Moving λ_max without moving the loss



Rationale. Sharpness is not invariant to reparametrization. If a parameter group a is multiplied by c and its partner b by 1/c, the network computes the same function, so the loss is unchanged, but the Hessian block of a is scaled by $1/c^2$ and the block of b by c^2. The pairs in this architecture are:

- the output head `lm_head` and the final-norm gain (logits = lm_head(norm(x))),
- `up_proj` and `down_proj` in each MLP (the gate is nonlinear, but `up_proj` multiplies linearly),
- `v_proj` and `o_proj` in each attention layer,
- `q_proj` and `k_proj`, which feed RMSNorms (QK-norm), so rescaling them changes nothing.

Hand rescaling of the final AdamW checkpoint

Rescaling	Probe loss	λ_max	× reference	embedding	attention	qk_norm	mlp	norm	head
reference	2.9474	5.967	1.00	0.033	1.208	0.059	0.764	0.475	5.642
head ×0.25, final norm ×4	2.9474	90.547	15.17	0.033	1.208	0.059	0.764	0.236	90.266
head ×0.5, final norm ×2	2.9474	22.855	3.83	0.033	1.208	0.059	0.764	0.278	22.566
head ×4, final norm ×0.25	2.9474	5.506	0.92	0.033	1.208	0.059	0.764	4.940	0.353
`up_proj` ×0.25, `down_proj` ×4	2.9474	6.749	1.13	0.033	1.208	0.059	3.072	0.475	5.642
`v_proj` ×0.25, `o_proj` ×4	2.9474	16.864	2.83	0.033	16.220	0.059	0.764	0.475	5.642
`q_proj` ×0.25, `k_proj` ×0.25	2.9474	7.098	1.19	0.033	6.754	0.059	0.764	0.475	5.642

- **The loss does not move** (2.9474 to four digits in every row) while λ_max changes by up to 15×.
- **The block that is scaled down gets sharper by about $1/c^2$.** The head goes 5.64 to 22.6 (c = 0.5, ×4) and to 90.3 (c = 0.25, ×16).
- **The whole-model λ_max follows the sharpest block.** MLP rescaling raises the MLP block 4× (0.76 to 3.07) but the model's λ_max moves only 1.13× because the head is still the sharpest block (5.64). Scaling the head up (×4) removes the head from the top (0.353), but λ_max only falls to 5.5 because the norm block has risen to 4.94.
- **At least a 2× change at fixed loss** is reached by the head ×0.5 and ×0.25 and the `v_proj`/`o_proj` rescalings.

Training with a sharper or flatter head from the start

`head_init_scale = c` (a new option in `train.py`) multiplies the output head by c at initialization and divides the final-norm gain by c, so the model is identical at initialization. Everything else is the default recipe (three runs, one seed each).

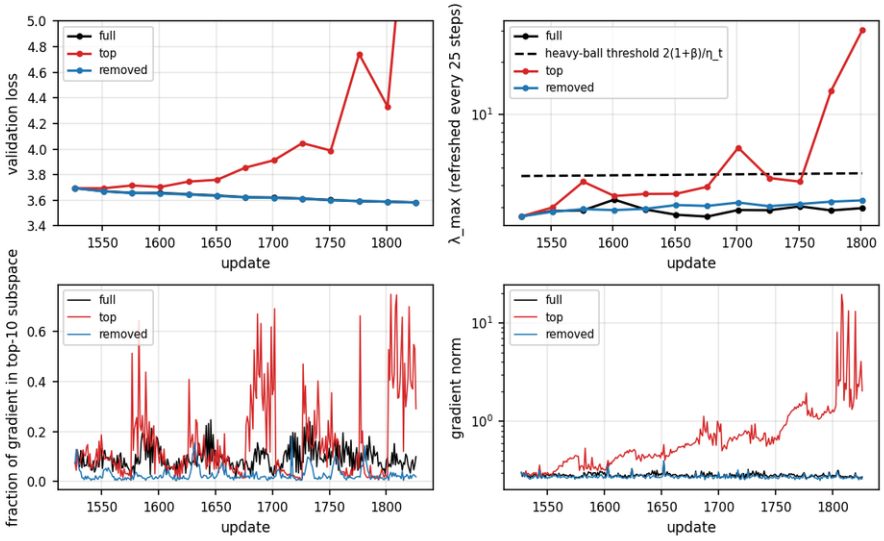
head_init_scale	Final val loss	Probe loss	λ_max	× default	head λ_max
default (1)	2.925–2.931 (three seeds)	2.9474	5.967	1.00	5.642
0.25	**2.9257**	2.9464	13.399	2.25	12.917
4	2.9320	2.9529	2.593	0.43	2.240
0.0625	2.9602	2.9805	290.294	48.65	289.302

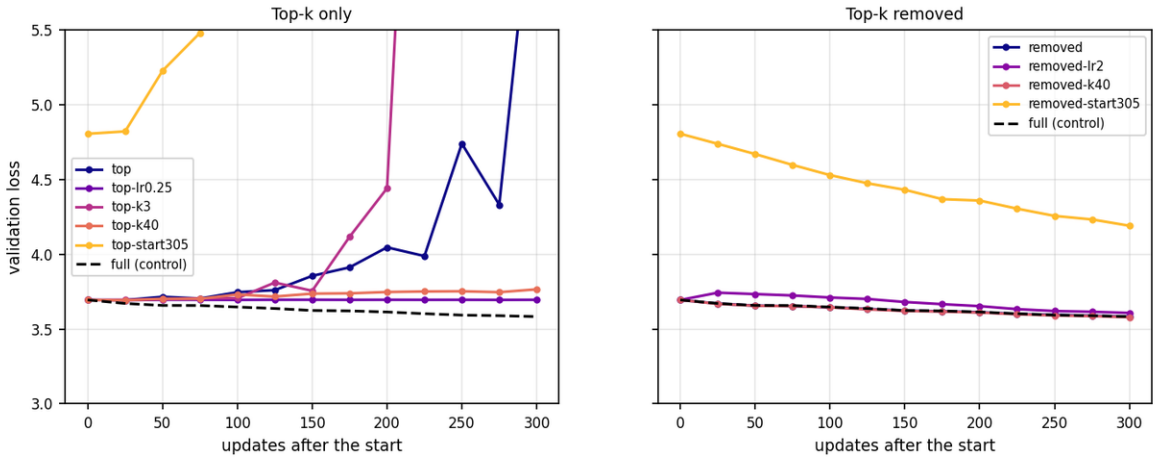
- **head_init_scale = 0.25 meets both criteria:** the final validation loss (2.9257) is inside the 2.925–2.931 range of the default recipe and λ_max is 2.25× larger (13.4 against 4.5–6.0).
- **head_init_scale = 4** lowers λ_max 2.3× (to 2.6) at a loss of 2.932, just above the range (by 0.001), so it is borderline on the loss criterion.
- **head_init_scale = 0.0625** makes the head 48× sharper but costs 0.03 in loss (2.960), outside the range, so there is a limit to how far this can go for free.
- **Training only partly keeps the initial scale.** Starting with the head at 0.25 gives a head λ_max of 12.9, not the 90 of the same rescaling applied to the finished checkpoint, which is consistent with AdamW growing the small head back toward the scale it would otherwise reach (updates of size η per weight regardless of the weight's size). The effect is still large and persistent.

What this means for comparing λ_max across runs. Two models with the same function and the same loss can differ in λ_max by an arbitrary factor, just by how the weights are scaled, and an initialization or optimizer choice can change that scale during training. A larger λ_max in one run does not mean the loss landscape around its solution is sharper in any function-space sense. Comparisons are only meaningful between runs with the same parametrization and initialization, or after normalizing for the weight scale (for example by weight norms or using a reparametrization-invariant measure).

(c) Training in and out of the top subspace

Starting point: the SGD-with-momentum checkpoint at 100M tokens (step 1526, LR about 0.85, β = 0.9), momentum zeroed, 300 updates, top-k basis refreshed every 25 updates. The "fraction of the gradient in the subspace" is the share of the squared gradient norm.





Job	Start step	Start val	Final val	Δ val	Mean share of gradient in top-k	Final λ_max / threshold
full	1526	3.694	3.583	−0.112	0.093	0.63
top-10 only	1526	3.694	6.706	+3.011	0.155	6.46
top-10 removed	1526	3.694	3.582	−0.112	0.028	0.70
top-10 only, LR × 0.25	1526	3.694	3.695	+0.001	0.024	0.16
top-3 only	1526	3.694	43.531	+39.836	0.094	135.30
top-40 only	1526	3.694	3.765	+0.071	0.136	0.62
top-10 only, start 305	305	4.807	45.148	+40.342	0.110	381.49
top-10 removed, LR × 2	1526	3.694	3.607	−0.087	0.035	1.09
top-40 removed	1526	3.694	3.579	−0.115	0.041	0.73
top-10 removed, start 305	305	4.807	4.191	−0.615	0.049	0.99

- **The gradient is not mostly in the top subspace here.** In the full update only about 9% of the squared gradient lies in the top-10 eigenspace (12% on the probe), far from the "mostly in the top few eigenvectors" of Gur-Ari et al. for this language model.
- **Removing the top-10 directions costs nothing.** The update with them removed trains exactly like the full update (3.582 against 3.583 after 300 updates), even though it discards about 9% of the gradient.
- **Moving only in the top-10 subspace is unstable.** The loss rises (3.69 to 6.7), the gradient norm grows by more than 10× and λ_max reaches 6.5 times the heavy-ball threshold $2(1+\beta)/\eta$_t by the end, so the run diverges as a quadratic would above the threshold. The full run stays below the threshold (λ_max is 0.63 of it at the end) and so does the removed run (0.70).
- **Top-only is a stability problem, not a lack of signal.** At LR × 0.25 the top-only run is stable (3.695), but it makes no progress at all (+0.001) in 300 updates, so steps along the sharpest directions neither help nor are safe at the base LR.
- **More directions is safer.** top-3 diverges fastest (final val 43.5), top-10 diverges more slowly (6.7), and top-40 is stable but drifts up slowly (+0.071). Starting earlier, at 20M tokens (step 305), the top-only update diverges even faster (45.1), and the removed update still trains (−0.615), with no full-update control there.
- **Changes that make training faster: none.** The removed-subspace update with k = 40 improves the loss by 0.115, against 0.112 for the full update, which is within noise, and a larger LR for the removed update (×2) is slower (−0.087), and puts λ_max just above the threshold (1.09).

Mental model. The top eigendirections are the steep walls of the river valley: they hold a small share of the gradient, the iterates bounce across them at the edge of stability, and following them alone makes the iterate oscillate and diverge. Progress is made along the many flat bulk directions that remain when the top-k are removed, which is why the removed update trains as fast as the full one. A larger LR does not make it faster (×2 is slower and brings λ_max to the threshold), so in these runs the sharp directions are not what limits the speed.

Synthesis

Sharpness (λ_{max}) is not a property of the function the model computes: rescaling matched pairs of parameters changes it 15× at an identical loss, and a different initialization scale changes it 2× at the same final loss, so λ_{max} should only be compared at matched parametrization. Along the d8 AdamW run the sharpest block moves from the input embedding at initialization to the output head from 10M tokens on, and the spectrum is a spike at zero with a few outliers, almost no negative curvature, and about 20% of $\|H\|_F^2$ in the top eigenvector at the end. In SGD with momentum the top-10 eigenvectors carry only about 9% of the gradient; the optimizer's progress comes from the bulk of flat directions, and following the sharp ones alone destabilizes training.